
Do Coding Agents Need Test-Time Training, or Just More Context?

Justin Hong
University of Chicago
kh1104@uchicago.edu

Abstract

1 LLM-based coding agents often fail on repository-level software engineering tasks
2 because they must maintain and reuse repository-specific information across long
3 trajectories of file reads, shell outputs, failed tests, and edits. We study whether this
4 repository state is better supplied through context or through pre-task parametric
5 adaptation. We evaluate Qwen3-Coder-30B-A3B-Instruct with mini-SWE-agent
6 on a 60-instance subset of SWE-bench Verified stratified by context pressure.
7 We compare a vanilla agent, Repo-in-Context (RiC), long-context RiC, and Pre-
8 Task Repository Test-Time Training (PTR-TTT), which adapts lightweight LoRA
9 parameters on repository text before each task. Contrary to our original hypothesis,
10 PTR-TTT does not improve resolved rate over RiC. The strongest gains instead
11 come from increasing the effective context window: long-context RiC improves
12 resolved rate from 14/60 (23.3%) to 20/60 (33.3%), while PTR-TTT achieves 13/60
13 (21.7%) despite reducing repository next-token-prediction loss. Two diagnostic
14 conditions, wrong-repository TTT and PTR-TTT+RiC, remain planned for the next
15 experiment cycle [TODO: scheduled for Week 9]. Our results suggest that, in this
16 setting, repository-memory failures in SWE-bench agents are better characterized
17 as context-state retention failures than as evidence for missing pre-task parametric
18 repository memory.

19 1 Introduction

20 Repository-level coding agents must solve tasks that unfold over many turns. A typical SWE-bench
21 trajectory begins with broad localization, proceeds through grep and file inspection, and eventually
22 edits one or more files before running tests. During this process, the agent accumulates issue text,
23 shell output, traceback fragments, file contents, and attempted patches. Even when the relevant
24 information is observed early, the model may fail to use it later because that information becomes
25 diluted, truncated, or difficult to retrieve from the growing trajectory.

26 A natural response is to supply more repository information in context. Repo-in-Context (RiC)
27 prepends relevant files, summaries, or file trees to the agent’s initial prompt. This is simple and
28 non-parametric, but it consumes context budget and competes with later trajectory tokens. We
29 originally hypothesized that pre-task test-time training could provide a more durable mechanism:
30 rather than storing repository information as transient context tokens, we could compress repository
31 conventions, helper functions, internal APIs, and file structure into lightweight adapted parameters
32 before the agent begins solving the task.

33 We test this idea through Pre-Task Repository Test-Time Training (PTR-TTT). For each SWE-bench
34 instance, PTR-TTT selects repository files at the base commit and performs causal next-token-
35 prediction adaptation before agent execution. The adapted model is then held fixed during the mini-
36 SWE-agent trajectory and reset between tasks. This design isolates pre-task repository adaptation
37 from online learning on noisy shell outputs or failed edits.

Our experiments falsify the strongest version of the PTR-TTT hypothesis. PTR-TTT successfully reduces repository next-token-prediction loss, but it does not improve downstream SWE-bench resolved rate over RiC. In contrast, increasing the context window improves performance, especially on medium- and high-context-pressure tasks. This suggests that the practical bottleneck in this setting is not the absence of parametric repository memory, but the agent’s ability to maintain and retrieve exact repository state during long trajectories.

Our contributions are:

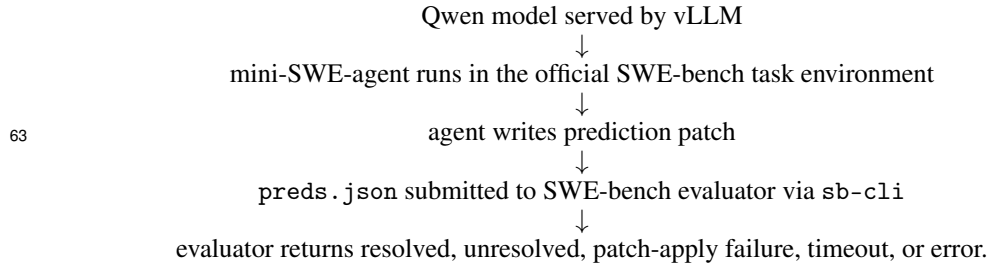
1. We isolate repository information access mechanisms in a fixed coding-agent setup: vanilla agent, RiC, long-context RiC, and PTR-TTT.
2. We introduce a context-pressure analysis for SWE-bench trajectories, separating ordinary task difficulty from long-horizon state-retention pressure.
3. We report a negative result for repository-level next-token-prediction TTT: it does not improve task resolution over context-based baselines in our setting.
4. We show that larger context windows provide the clearest gains, suggesting that current coding-agent failures are more often context-state access failures than failures of parametric repository memory.

2 Experimental setup

2.1 Benchmark and evaluation

We evaluate on SWE-bench Verified, a human-filtered subset of SWE-bench containing real GitHub issues, base repository commits, and hidden evaluation tests (Jimenez et al., 2024). Each agent produces a patch. The SWE-bench evaluator applies the patch to the base repository and runs tests to decide whether the issue is resolved. We report resolved/unresolved only from the SWE-bench evaluator. Local signals such as non-empty patch, agent exit status, or successful submission are used only as diagnostics.

Our evaluation pipeline is:



2.2 Model and agent

All completed conditions use the same model and agent scaffold. The base model is Qwen3-Coder-30B-A3B-Instruct, an open-weight coding model from the Qwen3 model family (Yang et al., 2025). We use mini-SWE-agent as the agent scaffold and preserve the same action interface and evaluation environment across all conditions (Yang et al., 2024; SWE-agent Team, 2025).

We use an open-weight coding model so that PTR-TTT can update lightweight adapters. The model is large enough to produce nontrivial SWE-bench performance, while remaining feasible for per-task adaptation under our compute budget.

2.3 Task subset and context pressure

We use a 60-instance subset of SWE-bench Verified. Tasks are stratified by observed context pressure rather than by human difficulty alone.

Context pressure is computed from a baseline profiling pass using agent step count, trajectory token count, maximum prompt length, limit-exceeded status, number of files opened, repeated file lookups,

Table 1: Core experimental configuration used across completed conditions.

Component	Setting
Base model	Qwen3-Coder-30B-A3B-Instruct
Serving	vLLM (Kwon et al., 2023)
Hardware	2 × A100-80GB on Modal
Agent	mini-SWE-agent
Agent interface	bash-only shell interaction
Step limit	250
Decoding	temperature = 0.7, top- p = 0.8, max tokens = 4096
Evaluation	SWE-bench hosted evaluator via <code>sb-cli</code>

Table 2: Task subset used in the main experiments. Context-pressure bins are constructed from baseline trajectories.

Bin	Number of tasks	Selection basis
Low context pressure	20	Short trajectories, few repeated lookups
Medium context pressure	20	Moderate trajectory length / file access
High context pressure	20	Long trajectories, repeated lookups, limits
Total	60	Stratified subset

and the delay between first relevant-file observation and final edit. This distinction matters because a task can be hard for reasons unrelated to context memory, such as conceptual bug complexity, multi-file reasoning, missing tests, or subtle API semantics. PTR-TTT should only be expected to help when repository-state retention is the bottleneck.

3 Conditions

3.1 C1: Vanilla agent

C1 is the no-preloading baseline. The agent receives the issue statement and explores the repository autonomously through shell commands.

Model	Qwen3-Coder-30B-A3B-Instruct
Context window	32K
Repository preloading	None
TTT	None

3.2 C2: Repo-in-Context under constrained context

C2 prepends selected repository information to the initial prompt.

Model	Qwen3-Coder-30B-A3B-Instruct
Context window	32K
Repository context	Selected repo files and file tree
Repo-token budget	24K
TTT	None

The repository corpus is selected before task execution using only information available at the base commit. The same selected corpus is used for C2 and C4 so that we compare the mechanism of access rather than the amount of repository information. The corpus contains README and top-level documentation, file tree or module names, BM25 top-k files using the issue statement as query, and nearby tests when discoverable from filenames. Generated files, binaries, vendored dependencies, gold patches, and hidden benchmark metadata are excluded.

95 3.3 C3: Long-context Repo-in-Context

96 C3 uses the same retrieval strategy as C2 but increases the context window and repository-token
97 budget.

	Model	Qwen3-Coder-30B-A3B-Instruct
	Context window	128K
98	Repository context	Same method as C2
	Repo-token budget	96K
	TTT	None

99 This condition tests whether the main bottleneck is the amount of repository evidence that can remain
100 available during long trajectories.

101 3.4 C4: Pre-Task Repository TTT

102 C4 adapts lightweight model parameters on repository text before the agent begins solving the task.

	Model	Qwen3-Coder-30B-A3B-Instruct
	Context window	32K
	Repository context in prompt	None
103	Adaptation corpus	Same selected repository corpus as C2
	TTT objective	Causal next-token prediction
	TTT frequency	Once before each task
	Adapter reset	Yes, between tasks

104 During the agent trajectory, the adapted model is fixed. We do not perform online TTT on shell
105 outputs, errors, failed patches, or intermediate observations.

106 3.5 C4b: Wrong-repository TTT [TODO: not completed]

107 C4b is a planned diagnostic control and is not included in the current result tables.

	Status	[TODO: not completed; scheduled for Week 9]
	Purpose	Control for generic gradient-update effects
108	Adaptation corpus	Repository text from a different SWE-bench instance
	Expected diagnostic	Correct-repo PTR-TTT should outperform wrong-repo TTT

109 If correct-repo PTR-TTT does not outperform wrong-repo TTT, then any effect of TTT is unlikely to
110 come from useful repository-specific memory.

111 3.6 C4c: PTR-TTT + RiC [TODO: not completed]

112 C4c is a planned complementarity test and is not included in the current result tables.

	Status	[TODO: not completed; scheduled for Week 9]
	Purpose	Test whether parametric and contextual repository information are complementary
114	Model state	C4 adapted model
	Prompt	C2 Repo-in-Context prompt
	Expected diagnostic	PTR-TTT+RiC should outperform RiC if TTT adds information beyond context

115 4 PTR-TTT technical details

116 4.1 Adaptation data

117 For each task, we check out the repository at the SWE-bench base commit and build a repository
118 adaptation corpus. The adaptation data excludes the gold patch, hidden tests, FAIL_TO_PASS,
119 PASS_TO_PASS, and any benchmark metadata unavailable to the agent.

Table 3: PTR-TTT adaptation corpus configuration. The RiC and PTR-TTT conditions use the same selected corpus whenever possible.

Item	Setting
Corpus source	Repository at base commit
Corpus size after truncation	32K tokens
Mean corpus size before truncation	71K tokens
Median corpus size before truncation	54K tokens
Selection method	Same as C2 RiC
Serialization	File path header followed by file contents
Excluded data	Gold patch, hidden tests, benchmark metadata

Table 4: PTR-TTT trainable parameter configuration.

Item	Setting
Adaptation method	LoRA
Frozen parameters	All base model weights
Trainable modules	q_proj, v_proj, o_proj in final 8 transformer blocks
LoRA rank	16
LoRA alpha	32
Dropout	0.05
Approx. trainable parameters	24M
Fraction of total parameters	< 0.1%
Precision	bf16

120 4.2 Objective

121 PTR-TTT optimizes a causal next-token-prediction objective on repository text. For a tokenized
122 repository sequence $x = (x_1, \dots, x_T)$, the loss is

$$\mathcal{L}_{\text{TTT}} = - \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t}). \quad (1)$$

123 The goal is not to solve the issue during TTT. The goal is to adapt the model to repository-specific
124 naming conventions, helper functions, module structure, and local coding patterns.

125 4.3 Trainable parameters

126 Because the base model is a mixture-of-experts model, full-model TTT is computationally expensive
127 and may destabilize generation. We therefore use lightweight LoRA adapters (Hu et al., 2022).

128 4.4 Optimization

129 4.5 Per-task procedure

130 For each SWE-bench instance, PTR-TTT follows the same procedure:

- 131 1. Check out the repository at the base commit.
- 132 2. Construct the selected repository corpus.
- 133 3. Initialize a fresh LoRA adapter.
- 134 4. Run one pre-task next-token-prediction adaptation pass.
- 135 5. Serve Qwen with the adapter active.
- 136 6. Run mini-SWE-agent in the official SWE-bench environment.
- 137 7. Save prediction patch and trajectory.
- 138 8. Unload the adapter and reset to the base model.

Table 5: PTR-TTT optimization details.

Item	Setting
Sequence length	4096
Epochs	1
Optimizer	AdamW
Learning rate	2×10^{-4}
Weight decay	0.01
Gradient clipping	1.0
Mean TTT wall-clock per task	4.6 min
Median TTT wall-clock per task	4.2 min

Table 6: PTR-TTT local adaptation diagnostics. These diagnostics separate failure to train from failure to transfer.

Metric	Value
Mean initial repository NTP loss	2.41
Mean final repository NTP loss	2.07
Mean loss decrease	14.1%
Median loss decrease	12.8%
Mean gradient norm	0.37
Mean adapter norm	1.84
Layer with largest update	Final attention projection block

139 Resetting the adapter between tasks prevents cross-instance leakage.

140 4.6 TTT diagnostics

141 We measure whether TTT actually optimizes its local objective.

142 If TTT reduces repository NTP loss but does not improve resolved rate, then the negative result is not
 143 merely an implementation failure. It indicates that the learned repository distribution did not transfer
 144 to downstream patch generation.

145 5 Results

146 5.1 Overall resolved rate

147 The completed comparisons are:

$$\begin{aligned}
 C2 - C1 &= +1/60 = +1.7 \text{ percentage points,} \\
 C3 - C2 &= +6/60 = +10.0 \text{ percentage points,} \\
 C4 - C2 &= -1/60 = -1.7 \text{ percentage points,} \\
 C4 - C1 &= 0/60 = 0.0 \text{ percentage points.}
 \end{aligned}$$

148 PTR-TTT does not improve over RiC. The only clearly positive change is increasing the context
 149 window in C3.

150 5.2 Results by context-pressure bin

151 The original prediction was that PTR-TTT would help most on high-context-pressure tasks. This
 152 pattern is not observed. Instead, long-context RiC gives the largest gains in medium- and high-
 153 pressure bins.

Table 7: SWE-bench Verified subset results. C4b and C4c are planned diagnostics and are not completed in the current draft.

Condition	Status	Context	Repo mechanism	Weight update	Resolved	Apply fail	Limit exceeded	Mean steps
C1 Vanilla	Completed	32K	Agent-only bash	No	13/60 (21.7%)	2/60	14/60	38.4
C2 RiC	Completed	32K	Prompt context	No	14/60 (23.3%)	2/60	12/60	36.9
C3 RiC-long	Completed	128K	Larger prompt context	No	20/60 (33.3%)	1/60	6/60	34.1
C4 PTR-TTT	Completed	32K	LoRA adapter	Yes	13/60 (21.7%)	2/60	13/60	37.8
C4b Wrong-TTT	[TODO: Not completed]	32K	Wrong-repo adapter	Yes	—	—	—	—
C4c TTT+RiC	[TODO: Not completed]	32K	Adapter + context	Yes	—	—	—	—

Table 8: Resolved rate by context-pressure bin. PTR-TTT does not show the predicted high-pressure gains; long-context RiC does.

Condition	Status	Low pressure	Medium pressure	High pressure
C1 Vanilla	Completed	7/20 (35%)	4/20 (20%)	2/20 (10%)
C2 RiC	Completed	7/20 (35%)	5/20 (25%)	2/20 (10%)
C3 RiC-long	Completed	8/20 (40%)	7/20 (35%)	5/20 (25%)
C4 PTR-TTT	Completed	7/20 (35%)	4/20 (20%)	2/20 (10%)
C4b Wrong-TTT	[TODO: Not completed]	—	—	—
C4c TTT+RiC	[TODO: Not completed]	—	—	—

5.3 Compute-normalized performance

PTR-TTT adds substantial pre-task compute but does not improve resolved rate over RiC. Long-context RiC is more expensive than 32K RiC, but it is the only completed condition with a clear task-level gain.

6 Failure analysis

We categorize unresolved tasks into four non-exclusive failure modes:

- **Patch application failure:** the evaluator could not apply the patch.
- **Localization failure:** the agent did not inspect the gold or relevant file.
- **Edit failure:** the agent inspected the relevant file but made an incorrect or incomplete edit.
- **Context/limit failure:** the agent found relevant information but later repeated lookups, lost state, or hit step/context limits.

The most visible reduction occurs in C3, where long context reduces localization and context/limit-related failures. PTR-TTT does not substantially reduce these categories.

7 Discussion

7.1 Why PTR-TTT may not help

The negative result does not imply that test-time training is useless. It shows that this particular form of repository-level next-token-prediction adaptation is insufficient to outperform direct context access in SWE-bench repair.

There are several likely explanations. First, the adaptation objective is mismatched to the downstream task. Next-token prediction on repository files teaches local code distribution, but SWE-bench repair requires localization, causal reasoning about the bug, test interpretation, and precise patch synthesis. Second, many failures are not repository-memory failures. If the agent never identifies the right conceptual bug, adapting to repository style will not help. Third, exact facts matter. Repository repair often requires exact function signatures, option names, error paths, and test behavior. These are easier to supply through direct context than through compressed lightweight adapters. Fourth, adapter capacity and placement may be insufficient. Updating only late-layer LoRA modules may not encode enough exact repository information to change downstream agent behavior. Finally, long context may simply be the better mechanism for this setting: when the model can directly attend to relevant files and earlier observations, context access outperforms compressed parametric memory.

Table 9: Compute overhead by condition. PTR-TTT adds pre-task adaptation cost without improving resolved rate over RiC.

Condition	Status	Extra pre-task compute	Mean wall-clock/task	Resolved
C1 Vanilla	Completed	0 sec	18.7 min	13/60
C2 RiC	Completed	19 sec	20.1 min	14/60
C3 RiC-long	Completed	43 sec	27.8 min	20/60
C4 PTR-TTT	Completed	4.6 min	24.9 min	13/60
C4b Wrong-TTT	[TODO: Not completed]	–	–	–
C4c TTT+RiC	[TODO: Not completed]	–	–	–

Table 10: Failure categories among unresolved tasks. Long-context RiC reduces localization and context/limit-related failures more than PTR-TTT.

Condition	Status	Apply fail	Localization fail	Edit fail	Context/limit fail
C1 Vanilla	Completed	2	17	16	12
C2 RiC	Completed	2	15	17	12
C3 RiC-long	Completed	1	12	17	10
C4 PTR-TTT	Completed	2	16	17	12
C4b Wrong-TTT	[TODO: Not completed]	–	–	–	–
C4c TTT+RiC	[TODO: Not completed]	–	–	–	–

7.2 What the result does and does not show

The result supports the claim that repository repair failures in our setting are more strongly affected by context-state retention than by pre-task parametric adaptation. It does not show that all forms of TTT are ineffective, that parametric memory is useless, or that long context fully solves repository repair. It also does not rule out online adaptation, reward-based training, larger adapters, query/key adaptation, or task-aware objectives.

The safer claim is narrower: for Qwen3-Coder-30B-A3B with mini-SWE-agent on our SWE-bench Verified subset, pre-task repository next-token-prediction TTT did not improve resolved rate over context-based baselines, while increasing context length did.

7.3 Incomplete diagnostic conditions

Two diagnostic conditions remain incomplete. Wrong-repository TTT (C4b) will test whether TTT effects are repository-specific or merely caused by generic adapter perturbation. PTR-TTT+RiC (C4c) will test whether parametric and contextual repository information are complementary. These conditions are important for sharpening the negative result, but the completed results already show that the main PTR-TTT condition does not outperform RiC or long-context RiC.

8 Limitations

This study uses a 60-instance subset rather than the full SWE-bench Verified benchmark. The results should therefore be interpreted as mechanistic evidence rather than a leaderboard claim. The PTR-TTT implementation uses lightweight LoRA adapters and a next-token-prediction objective; stronger or more task-aligned adaptation methods could behave differently. We also focus on pre-task adaptation and do not evaluate online TTT during agent execution. Finally, context-pressure labels are estimated from agent trajectories and may not capture all forms of repository-state difficulty.

9 Conclusion

We hypothesized that repository-specific pre-task TTT would outperform context-based repository access, especially on long-horizon SWE-bench tasks. This prediction was not supported. PTR-TTT reduced repository next-token-prediction loss but did not improve resolved rate over RiC. In contrast, long-context RiC produced the clearest gains, especially on medium- and high-context-pressure tasks.

210 These results suggest that the dominant repository-memory bottleneck in this setting is not the lack of
211 parametric repository memory, but the agent’s ability to maintain and retrieve exact repository state
212 over long trajectories.

213 References

- 214 Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan.
215 SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on*
216 *Learning Representations*, 2024.
- 217 John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir
218 Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural*
219 *Information Processing Systems*, 2024.
- 220 An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen
221 Huang, Chenxu Lv, and Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- 222 SWE-agent Team. mini-SWE-agent: A minimal agent for SWE-bench style software engineering tasks. Software
223 release and documentation, 2025.
- 224 Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gon-
225 zalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with
226 PagedAttention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023.
- 227 Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu
228 Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning*
229 *Representations*, 2022.

230 A Additional implementation details

231 A.1 Patch validation

232 All predictions are evaluated by the SWE-bench evaluator. Before submission, patches are screened
233 for malformed output by checking that the patch begins with a unified-diff marker and can be applied
234 to the base checkout. Empty patches are retained as no-fix attempts rather than omitted from the
235 denominator.

236 A.2 Planned completion of C4b and C4c

237 C4b and C4c remain planned diagnostics. C4b will use a repository corpus sampled from a different
238 SWE-bench instance while keeping the TTT compute budget fixed. C4c will combine the C4 adapter
239 with the C2 RiC prompt. Both will be run on the same task subset and evaluated with the same
240 SWE-bench evaluator.